

Embedded Knowledge Base

ESP32 车载性能仪 • 个人技术知识库

工作区：e:\esp32\esp32-imu

生成日期：2026-06-10

这份文档是写给半年后的自己的

重新打开这个仓库时，只读这一份文档就能想起来：当时做了什么、为什么这么设计、数据怎么流、核心技术是什么、踩过哪些坑。

关于工作区范围：当前工作区只有一个固件项目（esp32-imu），外加随附文档（GPS 升级分析、面试归档、接线图、构建指南）和 cad/ 里的 OpenSCAD 外壳模型与 Gerber 压缩包。本文档据此组织。

目录

1	项目全景图	3
1.1	项目名称：ESP32 车载性能仪（“Dragy 风格” Performance Box）	3
1.1.1	项目背景	3
1.1.2	项目目标	3
1.1.3	技术栈	3
1.1.4	项目一句话总结（适合放简历）	3
1.1.5	项目目录结构分析	4
2	系统架构设计	4
2.1	系统由哪些模块组成	4
2.2	为什么这样拆分模块	5
2.3	模块之间如何协作（数据链路）	5
3	项目运行流程	5
3.1	上电之后发生什么（初始化流程）	5
3.2	主循环流程（任务调度）	6
3.3	中断流程	6
3.4	状态机流程	6
4	核心模块解析	7
4.1	模块一：MPU6050 IMU 驱动（mpu6050.cpp）	7
4.2	模块二：GPS 模块（gps_module.cpp）	7
4.3	模块三：性能分析器 + 摩托车模式（performance_analyzer.cpp）	7

4.4	模块四：录音模块（audio_recorder.cpp）	8
4.5	模块五：SD 记录（sd_logger.cpp）	8
4.6	模块六：Web 服务器（在 main.cpp 内）	8
5	技术知识提取	9
5.1	知识点 1：I ² C（双路总线设计）	9
5.2	知识点 2：振动环境下的三级级联滤波（本项目灵魂）	9
5.3	知识点 3：互补滤波（Complementary Filter）	9
5.4	知识点 4：GPS-IMU 速度融合 + 动态信心度	9
5.5	知识点 5：GPS 零漂抑制（三层）	10
5.6	知识点 6：ADC + 定时器中断采样（录音）	10
5.7	知识点 7：WAV 文件格式	10
5.8	知识点 8：SPIFFS vs SD 卡	10
5.9	知识点 9：PMTK / NMEA 协议	10
5.10	知识点 10：HTTP / WiFi SoftAP	11
6	设计决策分析	11
7	调试与踩坑记录	11
7.0.1	脱节细节（坑 8 展开）	12
7.0.2	潜在风险（代码审查角度，未必触发）	12
8	工程实践总结	12
8.0.1	做得好的地方	12
8.0.2	可以改进的地方	13
9	项目复盘	13
9.0.1	最大的亮点	13
9.0.2	最大的难点	13
9.0.3	学到的知识	13
9.0.4	如果重新设计会怎么做	13
9.0.5	下一步可以扩展什么	13
10	个人知识地图	13

1 项目全景图

1.1 项目名称：ESP32 车载性能仪（“Dragy 风格” Performance Box）

项目背景

想用便宜的开源硬件，自己做一台摩托车/汽车的车载性能记录仪，对标商业产品 Dragy（一种 GPS 性能盒子，专测 0-100 km/h、0-400m、刹车距离）。商业 Dragy 要 2000 元，而一套 ESP32 + MPU6050 + GT-U7 GPS 的料才几十块。

核心挑战不是“读个传感器显示出来”，而是：

- 传感器装在摩托车上，发动机振动极烈，MPU6050 原始数据被 20~50 Hz 机械振动淹没。
- GT-U7 是 30 元级老 GPS 模块，静止零漂 ± 2 km/h，10 Hz 更新，低速段噪声比信号还大。
- 要在这种“脏数据”上算出能看的 G 力、倾角、加速测试成绩。

项目真正的价值在于用软件算法把廉价硬件的潜力榨干。

项目目标

1. 实时显示速度 / G 力（纵向、横向、垂直）/ 压弯倾角 / 加速测试成绩。
2. 自动识别“开始骑行 / 停车”，自动开始/停止数据记录与录音。
3. 通过 WiFi 热点 + 网页把数据推到手机浏览器（无需 App）。
4. 专门的摩托车模式：压弯角度、翘头/翘尾检测、弯道半径估算。
5. 数据落盘（SPIFFS 录音 / SD 卡 CSV 行车数据）。

技术栈

维度	选型
芯片	ESP32-WROOM-32（双核 240MHz，板载 WiFi）
框架	Arduino Framework（基于 ESP-IDF）
操作系统	无 RTOS 显式使用，裸机 <code>loop()</code> + 软件分时调度（底层是 FreeRTOS，但未直接用任务/队列）
构建系统	PlatformIO（ <code>platformio.ini</code> ）
语言	C++（Arduino 风格）+ 前端 HTML/CSS/JS（ <code>data/index.html</code> ）
传感器	MPU6050（6 轴 IMU）、GT-U7（GPS，UBX-M8030 芯片）、模拟麦克风
显示	SSD1306 OLED 128×64（I ² C）
协议	I ² C（双路）、UART（GPS）、SPI（SD 卡 HSPI）、HTTP（Web）、NMEA/PMTK
关键库	Adafruit MPU6050 / SSD1306 / GFX、TinyGPSPlus、ArduinoJson
机械	OpenSCAD 参数化外壳（ <code>cad/</code> ）

项目一句话总结（适合放简历）

简历版

基于 ESP32 + MPU6050 + GT-U7 GPS 自研车载性能仪，通过三级级联滤波抑制发动机振动、GPS-IMU 互补滤波融合与动态信心度加权算法，将 30 元 GPS 模块的静止零漂从 ± 2.5 km/h 压到 ± 0.8 km/h；实现 0-100 km/h 加速测试、压弯倾角/翘头检测，并通过 WiFi 热点 + 网页实

时可视化。

项目目录结构分析

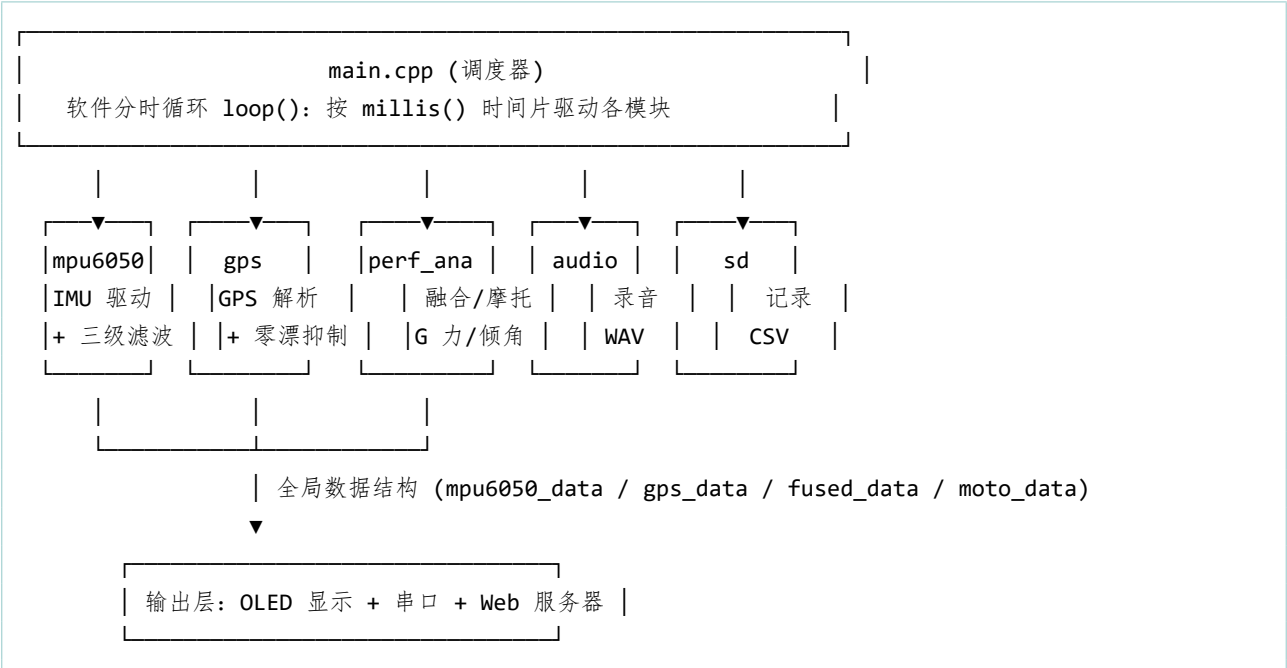
```
esp32-imu/
├─ platformio.ini      # PlatformIO 工程配置：板型、库依赖、串口
├─ src/                # 固件源码（核心）
│   ├── main.cpp       # 主程序：setup/loop、Web 服务器、OLED 显示、调度
│   ├── mpu6050.h,cpp  # IMU 驱动 + 三级振动滤波 + 姿态角计算
│   ├── gps_module.h,cpp # GPS 解析 + 零漂抑制 + GPS-IMU 融合 + 性能测试
│   ├── performance_analyzer.* # G 力计算 + 摩托车模式（倾角/弯道/翘头）
│   ├── audio_recorder.h,cpp # ADC 麦克风录音 → WAV，自动骑行触发
│   └─ sd_logger.h,cpp  # SD 卡 CSV 行车数据记录（HSPI）
├─ data/index.html     # SPIFFS 网页前端（手机端仪表盘）
├─ cad/               # 机械外壳（OpenSCAD 参数化模型 + STL）
├─ interview/         # LaTeX 版面试材料（main.tex/pdf）
├─ INTERVIEW_NOTES.md # [*] 面试技术答辩归档：踩坑/原理/量化效果（精华）
├─ GPS_UPGRADE_ANALYSIS.md # GPS 硬件升级对比分析
├─ GT-U7_ULTIMATE_OPTIMIZATION.md # GT-U7 软件压榨优化记录
├─ wiring_diagram.txt  # 接线图
├─ BUILD_GUIDE.md     # 编译/上传/串口/排错指南
└─ README.md          # 原始 README（[!] 已与代码脱节，见踩坑记录）
```

重点：真正的“知识密度”集中在 src/ 和 INTERVIEW_NOTES.md。

2 系统架构设计

2.1 系统由哪些模块组成

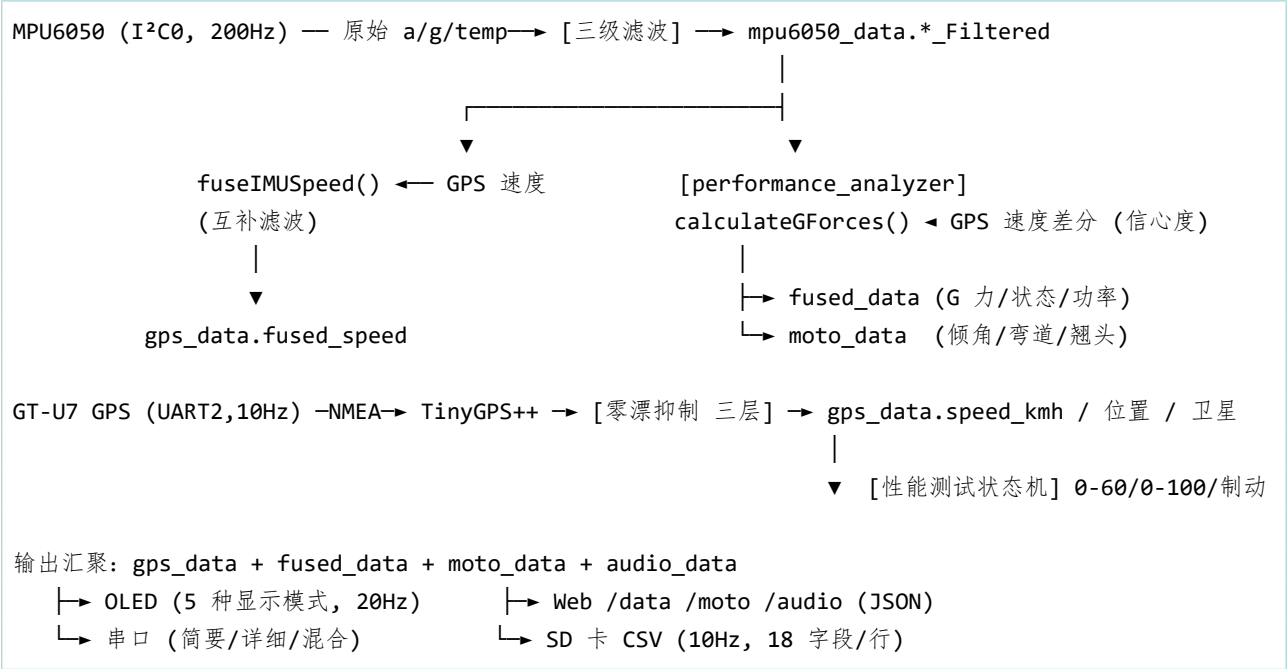
固件按“传感器采集 → 数据融合 → 输出”分层，五个功能模块围绕 main.cpp 的调度器协作：



2.2 为什么这样拆分模块

- 按物理传感器 + 按算法职责拆分。mpu6050/gps/audio/sd 对应四个外设;performance_analyzer 是纯算法层（不碰硬件，只消费别人的全局数据），把“业务逻辑”和“驱动”分开。
- 模块间用全局数据结构通信。mpu6050_data、gps_data、fused_data、moto_data 都是 extern 全局单例。这是嵌入式裸机常见做法：避免动态分配、零拷贝、任何模块随时能读最新值。代价是耦合度高、不可重入——但单线程 loop() 里没有竞态。
- 没有用 FreeRTOS 任务。所有“并发”靠 loop() 里基于 millis() 的时间片调度实现。对这种数据流明确、无阻塞操作的场景，裸机分时比多任务更简单、更可控。

2.3 模块之间如何协作（数据链路）



链路	频率	作用
IMU 采样 → 滤波	200 Hz	高频采集，为 GPS-IMU 融合提供积分基础
IMU → 速度融合	200 Hz	用加速度积分填补 GPS 帧间空白，GPS 帧到来时拉回漂移
GPS → 零漂抑制	10 Hz	把脏速度变成能用的速度，喂给所有下游
融合 → G 力/倾角	20 Hz	算出展示用的 G 力、压弯角、弯道信息
汇聚 → 输出	20/10 Hz	OLED/网页/串口（20Hz）、落盘（10Hz）

3 项目运行流程

3.1 上电之后发生什么（初始化流程）

setup()（main.cpp:713）的顺序是踩坑调整过的：

```
上电/Reset
→ Serial.begin(115200)          串口先起，方便看后续日志
→ pinMode(0, INPUT_PULLUP)      BOOT 按钮（GPIO0）做显示模式切换
→ 双路 I²C 初始化 I2C_MPU(21,22,400k) / I2C_OLED(18,19,400k)
```

```

→ scanI2CDevices()           扫总线确认 0x68/0x3C 在位（排错关键）
→ Init_mpu6050()             ±8G/±500°/s, DLPF=94Hz, 初始化滤波缓冲
→ initGPS()                  UART2 9600, PMTK 配 10Hz+ 精简 NMEA+SBAS
→ initPerformanceAnalyzer() / initMotorcycleMode() / initAudioRecorder()
→ [SD 卡初始化被注释跳过]    [!] 当前为“测试模式”（见踩坑记录）
→ display.begin()             OLED 启动, 开机画面 2 秒
→ SPIFFS.begin(true)          挂载文件系统（网页 + 录音）
→ WiFi.softAP("ESP32-IMU")   开热点, 固定 IP 192.168.4.1
→ server.on(...) + server.begin() 注册 6 个 HTTP 路由
→ display_mode=1, displayQRCode() 初始显示 WiFi 连接信息

```

3.2 主循环流程（任务调度）

loop() (main.cpp:835) 是系统心跳, 用 millis() 时间片做软件多任务:

```
loop()  server.handleClient();           ① Web 请求: 每圈都处理（非阻塞）  检测 BOOT 按钮 → 切换 display_mode (0~4)
```

关键设计：全程无 delay()

loop() 里绝对没有 delay()（除了按钮防抖那一处 200ms）。所有周期都用“时间戳差值比较”实现非阻塞，这样高频 IMU 不会被低频 GPS 拖慢——这是裸机分时调度能跑起来的前提。

3.3 中断流程

全项目只有一处硬件中断：录音采样定时器。教科书式的 ISR 最小化。

```

startRecording()
→ timerBegin(0, 80, true)      定时器 0, 80 分频 → 1MHz 计数
→ timerAlarmWrite(timer, 125)  125 tick = 125μs = 8kHz
→ timerAttachInterrupt(onSampleTimer)
    | 每 125μs 触发
    ▼
IRAM_ATTR onSampleTimer()      中断服务程序（放 IRAM 加速）    lastSample = adc1_get_raw()  只做最少的事：计
    | loop() 中
    ▼
processSample()                重活放主循环（ISR 外）          8 位量化 + 写缓冲 + 满 512 字节写 SPIFFS

```

原因——SPIFFS 写操作可能阻塞，绝不能在 ISR 里做。

3.4 状态机流程

项目里有三个独立状态机：

1) 录音状态机 (audio_recorder.cpp:296)

```

RECORDING_IDLE/STOPPED — 速度 ≥5km/h → RECORDING_ACTIVE
    ▲                               |
    |                               | 速度 < 2km/h 持续 5 秒
    └────────────────────────────────┘
(也可手动 start/stop/pause/resume)

```

2) 加速/制动测试状态机 (gps_module.cpp:337)

```

速度跨过 5km/h → 启动加速测试计时
├─ 跨 60km/h → 记 0-60 时间
├─ 跨 100km/h → 记 0-100 时间
└─ 跨 200km/h → 记 100-200, 结束
速度 ≥95km/h → 启动制动测试 — 速度 ≤5km/h → 记 100-0 时间

```

3) SD 自动记录状态机 (sd_logger.cpp:280): 速度 >5 km/h 开始记录, 停车 30 秒后结束。

4 核心模块解析

4.1 模块一: MPU6050 IMU 驱动 (mpu6050.cpp)

职责: 读 6 轴数据, 做振动滤波, 算姿态角。设计思路: 用 Adafruit 库读原始值, 但不信任原始值——叠加三级软件滤波。数据结构 MPU6050_DATA 同时保存“原始值”和“_Filtered 滤波值”两套: 下游算 G 力用滤波值, 算融合用原始值 (积分对延迟敏感)。

函数	作用
Init_mpu6050()	量程 $\pm 8\text{G}/\pm 500^\circ/\text{s}$, DLPF=94Hz; 失败时自动尝试 0x69 地址
ReadMPU6050()	读 a/g/temp → 滤波 → 算姿态角
FilterDataForVibration()	滑动平均 (10 点) + 一阶 IIR($\alpha=0.1$) 级联
CalculateAttitude()	atan2 算 Roll/Pitch ([!] Yaw 硬编码为 0)

4.2 模块二: GPS 模块 (gps_module.cpp)

职责: 解析 NMEA、抑制零漂、GPS-IMU 速度融合、跑加速/制动测试状态机。这是代码量最大、算法最密集模块 (756 行)。本质是“清洗 + 融合”流水线。

函数	作用
initGPS()	UART2 + PMTK 配置 (10Hz/精简 NMEA/SBAS/WAAS)
updateGPSData()	逐字节喂 TinyGPS++ → 三层零漂抑制 → 信号监控自恢复
fuseIMUSpeed()	GPS-IMU 互补滤波 (核心算法)
calibrateIMUBias()	静止时递推均值在线校准加速度偏置 (O(1) 内存)
interpolateSpeedAtTime()	速度阈值穿越线性插值, 把 10Hz 的 100ms 不确定度降到 10-20ms
calculateDistance()	Haversine 球面距离 (位置跳变检测、圈速)
saveSessionData/loadSessionData	用 ArduinoJson 把成绩存进 SPIFFS

4.3 模块三: 性能分析器 + 摩托车模式 (performance_analyzer.cpp)

职责: 纯算法层。把 IMU + GPS 融合成 G 力, 再衍生摩托专属指标。分两套数据 fused_data (通用 G 力/功率) 和 moto_data (压弯/翘头/弯道)。

函数	作用
<code>calculateGForces()</code>	动态信心度加权融合 GPS 差分加速度与 IMU；静止/动态两套逻辑
<code>calculateLeanAngle()</code>	互补滤波算压弯倾角（陀螺仪积分 + 加速度计修正， $\alpha=0.96$ ）
<code>detectWheelieStoppie()</code>	俯仰角 + 前向 G 力联合判断翘头/翘尾
<code>analyzeCorner()</code>	弯道进出检测 + 半径估算 $r = v^2 / (g \cdot G_{lat})$
<code>estimatePower()</code>	滚阻 + 风阻 + 加速力模型估算功率 (kW)

安装姿态硬编码

`calculateLeanAngle()` 里 `Gyro_Z_Filtered` 被当作“横滚”、`Gyro_X_Filtered` 当作“偏航”，因为传感器是垂直安装的（X 轴朝下承重）。坐标轴映射按这个安装姿态硬编码——换安装方向就要改。

4.4 模块四：录音模块（`audio_recorder.cpp`）

职责：定时器中断采集麦克风 ADC，存成 WAV 到 SPIFFS，骑行时自动录。数据结构：WAVHeader（44 字节标准 WAV 头，先占位后回填 size）、AUDIO_RECORDER_DATA（含录音状态机字段）。设计思路：8kHz/8bit/单声道——刻意压低规格省 SPIFFS 空间。中断只读 ADC，写盘在主循环。

4.5 模块五：SD 记录（`sd_logger.cpp`）

职责：把每个数据点（18 字段）写成 CSV 行到 SD 卡，方便事后用 Excel/Python 分析。设计思路：用 HSPI（而非默认 VSPI），MISO 改到 GPIO25 避开 GPIO12 启动冲突。每 100 个点 flush 一次（平衡掉电安全与写入性能）。

半成品功能

SD 初始化在 `setup()` 里被注释掉（“测试模式”），但 `loop()` 里仍调用 `checkAutoLogging` 和 `logDataPoint`。因 `initialized=false`，这些调用安全早退，不写盘。

4.6 模块六：Web 服务器（在 `main.cpp` 内）

职责：WiFi 热点 + HTTP 服务，把数据 JSON 化推给手机网页。

路由	作用
<code>/</code>	从 SPIFFS 返回 index.html（带 5 分钟缓存头）
<code>/data</code>	主数据 JSON：温度/G 力/角度/GPS/性能
<code>/moto</code>	摩托车模式 JSON：倾角/弯道/翘头/状态
<code>/audio</code>	录音状态
<code>/audio/control</code>	录音控制（start/stop/pause/list/...）
<code>/audio/download</code>	下载 WAV 文件

设计思路：用 `sprintf` 手拼 JSON（而非 ArduinoJson 序列化），固定 `char[]` 缓冲。原因——`/data` 高频轮询，手拼比构建 `JsonDocument` 快、内存可控。代价是缓冲区大小要算准，否则溢出。

5 技术知识提取

本部分只讲项目里真的用到、且有“为什么”的知识点。

5.1 知识点 1: I²C（双路总线设计）

MPU6050 与 OLED 都是 I²C 设备。项目用两个独立的 I²C 控制器：TwoWire(0) 给 MPU6050 (GPIO21/22)，TwoWire(1) 给 OLED (GPIO18/19)。共享其实可行（地址 0x68/0x3C 不冲突），分两路的好处是 OLED 刷新不会阻塞 IMU 的高频读取，代价是多占两个 GPIO。排错第一招：scanI2CDevices() 分别扫两条总线。

5.2 知识点 2: 振动环境下的三级级联滤波（本项目灵魂）

解决：发动机怠速 20~50 Hz 振动叠加，G 力跳变、姿态抖 $\pm 5^\circ$ 。

原始 ADC

- ① 硬件 DLPF setFilterBandwidth(94Hz) 砍掉 >94Hz，但保留快速动作响应
 - ② 滑动平均 FIR(窗口 10@200Hz) 等效窗 50ms，截止 ≈ 10 Hz，无相位累积，延迟 25ms
 - ③ 一阶 IIR 低通 ($a=0.1$) $y=a*x+(1-a)*y_{prev}$ ，截止 ≈ 3.5 Hz，O(1) 内存
- 最终 *_Filtered（等效截止 3~5Hz，振动衰减 >12dB）

关键权衡：

- DLPF 选 94Hz 而非 5Hz：陀螺仪要追摩托快速转向（可达 $90^\circ/\text{s}$ ），截止太低动态响应变迟钝。硬件留宽，靠软件补。
- IIR 不用 FIR：MCU 上 IIR 只存一个上次输出值（O(1)），FIR 要存整个窗口。加速度测量不做频谱分析，对相位延迟不敏感。
- $\alpha=0.1$ 怎么定： $f_c \approx \alpha f_s / (2\pi) \approx 0.1 \times 200 / 6.28 \approx 3.2$ Hz，再用串口波形微调。

量化效果：静止 G 力噪声 $\pm 0.15g \rightarrow \pm 0.01g$ ；姿态抖动 $\pm 5^\circ \rightarrow \pm 0.5^\circ$ ；代价 25~50ms 延迟。

5.3 知识点 3: 互补滤波（Complementary Filter）

解决：纯加速度计算姿态，动态时把“线加速度”误当“重力”，Pitch 虚假偏 $5\sim 10^\circ$ ；纯陀螺仪积分会漂移。原理：

$$\theta = \alpha (\theta_{prev} + \omega dt) + (1 - \alpha) \theta_{accel}$$

陀螺仪管高频（快、短期准），加速度计管低频（慢、长期准），互补。应用：摩托倾角（performance_analyzer.cpp:524）用 $\alpha=0.96$ 。注意姿态主路径（mpu6050.cpp）没用互补滤波，是已知缺口——G 力优先级更高，姿态次要。误区： α 不是越大越好。 $\alpha=0.98$ 时间常数 $\tau = \alpha \cdot dt / (1 - \alpha) \approx 0.25\text{s}$ ，这个“拉回速度”要和振动强度匹配着调。

5.4 知识点 4: GPS-IMU 速度融合 + 动态信心度

GPS 准但慢（10Hz、低速噪声大）；IMU 快（200Hz）但积分会漂。两个融合点：

1. 速度融合（fuseIMUSpeed，互补滤波）：200Hz 用加速度积分填 GPS 帧间空白，每个 GPS 帧（100ms）到来时按 GPS_CORRECTION_RATE=0.1 拉回 10% 漂移。
2. G 力融合（calculateGForces，动态信心度）：按实时信号质量算 imu_confidence = gps 稳定性 $\times$ 信号质量，动态分配 GPS 差分加速度与 IMU 的权重。

	本项目 (互补/自适应 IIR)	卡尔曼 (EKF)
计算量	极低, 适合 ESP32	矩阵运算重
调参	直观 (α 、死区)	要建模 Q/R 协方差
选择	[v]	改进方向

误区: 用 GPS 速度要用 Doppler 测速而非位置差分。TinyGPS++ 的 `gps.speed.kmph()` 解析的 RMC 速度本身就是 Doppler 速度, 精度比位置差分高 10 倍。项目用对了。

5.5 知识点 5: GPS 零漂抑制 (三层)

GT-U7 静止时输出 0.5~2 km/h 假速度。三层压制:

层 1: 20 点滑动平均 + 标准差分析	<code>std_dev<0.2</code> 且 <code>avg<</code> 动态阈值 $\rightarrow$ 判定稳定
层 2: 自适应 IIR (按速度段调 a)	$<0.5\text{km/h} \rightarrow a=0.92$ 强抑零漂; $>4\text{km/h} \rightarrow a=0.35$ 快响应不耽误计时
层 3: 最终死区	好信号 (≥ 8 星, HDOP <1.0) 阈值 0.8km/h , 否则 1.2km/h , 以下归零

判定静止需持续 1.5 秒 (15 次 $\times 100\text{ms}$)。这是精度 vs 响应的反复权衡。

5.6 知识点 6: ADC + 定时器中断采样 (录音)

为什么用 ADC1 的 GPIO34: ESP32 的 ADC2 与 WiFi 冲突 (WiFi 开启时 ADC2 不可用)。本项目全程开热点, 麦克风必须接 ADC1。GPIO34 是纯输入引脚, 最适合模拟输入。这是个容易踩、项目避开了的坑。采样: 硬件定时器 80 分频得 1MHz, 125 tick=125 μs =8kHz 触发, ISR 只 `adc1_get_raw` + 置标志。

5.7 知识点 7: WAV 文件格式

44 字节 RIFF 头 + PCM 裸数据。录音时先写占位头 (`size=0`), 结束时 `seek(0)` 回填真实 `dataSize/fileSize`。8bit 无符号 PCM, 中心在 128。

5.8 知识点 8: SPIFFS vs SD 卡

	SPIFFS (板载 Flash)	SD 卡 (外接)
容量	1-3MB	GB 级
用途	网页文件 + 录音 (小)	行车 CSV(大量)
接口	内部	SPI(HSPI)
项目状态	在用	半成品 (初始化被注释)

5.9 知识点 9: PMTK / NMEA 协议

GPS 用 NMEA 文本协议输出 (`$GPRMC/$GPGGA`), 用 PMTK 专有指令配置: `$PMTK220,100` 更新率 10Hz; `$PMTK314,...` 只输出 RMC+GGA (防 9600 波特率溢出); `$PMTK313,1` 启用 SBAS; `$PMTK301,2` DGPS 用 WAAS。

文档与代码不一致

GT-U7_ULTIMATE_OPTIMIZATION.md 说要把波特率提到 38400、用多星座, 但 `initGPS()` 实际代码仍是 9600 + 10Hz。优化文档是“设想”, 代码是“现状”。

5.10 知识点 10: HTTP / WiFi SoftAP

ESP32 开热点 (WiFi.softAP)，固定 IP 192.168.4.1，跑 WebServer。网页 JS 轮询 /data 拿 JSON。/data 返回 no-cache (实时数据)，/ (HTML) 返回 max-age=300 (静态可缓存)——缓存策略区分对了。

6 设计决策分析

决策 1: 裸机分时调度，而非 FreeRTOS 任务

loop() 用 millis() 时间戳差比较，全程无 delay()。优点：简单、可控、无竞态、无切换开销；缺点：任一环节阻塞拖垮全局，没用上第二个核。数据流明确、无长阻塞操作，裸机分时足够且心智负担小。

决策 2: 全局数据结构通信，而非消息传递

零拷贝、无动态分配、实现极简；强耦合、不可重入（但单线程无此问题）。嵌入式裸机标准做法。

决策 3: 环形缓冲区做滑动平均

定长数组 + 取模索引 + buffer_full 标志处理冷启动。O(1) 内存，无动态分配。

决策 4: 互补滤波而非卡尔曼

ESP32 算力和调参成本下，互补/自适应 IIR 性价比远高于 EKF。

决策 5: 双 I²C 总线

OLED 刷新不阻塞 IMU 读取；代价多占 2 个 GPIO。可单总线共享（地址不冲突）。

决策 6: 手拼 JSON 而非序列化库

热路径 (/data 高频) 选性能；冷路径 (saveSessionData) 用 ArduinoJson 选可维护——两种都用，按场景分。

决策 7: SD 卡用 HSPI + MISO 改 GPIO25

GPIO12 是 strapping 引脚（启动拉高致 flash 电压判断错、启动失败），把 MISO 挪到 GPIO25 避开——ESP32 经典硬件坑。

决策 8: 录音 8kHz/8bit

SPIFFS 只有几 MB，录的是骑行环境音/排气声，不需高保真，容量优先（8KB/s，5 分钟约 2.4MB）。

7 调试与踩坑记录

部分来自代码痕迹推断，部分已记录在 INTERVIEW_NOTES.md。

坑	原因 / 解决	状态
IMU 被发动机振动淹没	DLPF 默认 260Hz 等于没滤；三级级联滤波，等效截止 3~5Hz	[*] 已解决
GPS 静止零漂	多路径反射 + 几何误差；三层零漂抑制（均值 + 标准差/自适应 IIR/死区）	[*] 已解决
IMU 偏置致积分漂移	出厂零偏 $\pm 50\sim 150\text{mg}$ ；手动硬编码 + GPS 判静止时递推均值在线校准	[*] 部分解决
GPS 位置跳变	遮挡后重定位瞬跳数百米；Haversine 算帧间距离， $>1000\text{m}$ 拒绝	[*] 已解决
GPIO12 启动冲突 (SD)	strapping 引脚；MISO 挪到 GPIO25，用 HSPI	[*] 已规避
ADC2 与 WiFi 冲突 (麦克风)	WiFi 开启时 ADC2 读不到；麦克风接 ADC1 的 GPIO34	[*] 已规避
9600 下 10Hz NMEA 溢出	全量语句溢出丢帧；PMTK314 只留 RMC+GGA	[*] 已规避
文档与代码脱节	README/优化文档与代码多处对不上（见下）	[!] 待修
SD 记录是半成品	initSDCard() 被注释，loop() 仍调（initialized=false 安全早退）	[!] 待完成

脱节细节（坑 8 展开）

- README 写“共享 I²C 总线 GPIO21/19”，实际代码是双路 GPIO21/22 + GPIO18/19。
- README 的 lib_deps 写 electroniccats/MPU6050，实际 platformio.ini 用 adafruit/Adafruit MPU6050。
- GT-U7_ULTIMATE 说 38400 波特率/多星座，代码实际 9600/10Hz。

潜在风险（代码审查角度，未必触发）

- JSON 缓冲区：/data 用 char[750]、/moto 用 char[600] 手拼，字段多、%.8f 经纬度长，接近上限，加字段易溢出。建议加 snprintf 长度保护。
- 静止校准 static 局部变量：calculateGForces 里 gravity_bias_x 等用 static 且只累加不重置，长时间运行逻辑可能僵化（满 50 样本后固定）。

8 工程实践总结

做得好的地方

- 代码组织：按外设/算法清晰分模块，头文件声明 + cpp 实现规整。
- 错误处理：初始化大量带降级（MPU6050 试 0x68 失败试 0x69；OLED 失败仍继续；GPS 信号丢失自动热重启）。失败不死机、打印原因、降级运行的思路很成熟。
- 可观测性：串口三种打印模式可运行时切换；I²C 扫描、原始 NMEA 打印——调试钩子完备。
- 构建系统：PlatformIO 依赖锁版本，monitor_filters = esp32_exception_decoder 自动解崩溃栈。
- 配置管理：引脚、阈值、滤波系数都用 #define/const 集中在文件头。
- 场景化分层：热路径手拼 JSON 求性能，冷路径用 ArduinoJson 求可维护——分得清。

可以改进的地方

- 文档同步: README/优化文档已严重脱节 (坑 8)。
- 魔法数字: $-0.47/+0.5/-0.48$ 校准偏置散落多文件, 应抽成统一配置或自动校准。
- 缓冲区安全: 手拼 JSON 用 `sprintf` 无长度保护, 应换 `snprintf`。
- SD 功能收尾: 要么启用要么删, 别留半成品调用。
- 测试策略: 目前无任何自动化测试 (`test/` 只有空 README)。IMU 滤波、Haversine、零漂判定这些纯函数很适合 host 端单元测试。
- 持续集成: 无 CI, 可加 `pio ci` 在 push 时编译检查。

9 项目复盘

最大的亮点

把“算法”当成项目主角, 而非“点亮外设”。三级滤波 + GPS-IMU 融合 + 动态信心度这套组合, 把 30 元 GPS 的静止零漂从 ± 2.5 压到 ± 0.8 km/h (与专业设备差距从 20 倍缩到 4 倍, 成本差 67 倍)。

最大的难点

在脏数据上做可信的实时融合。难点不在写公式, 而在: 什么时候信谁、信多少 (动态信心度)、怎么不让滤波延迟拖垮测试计时 (自适应 α)。都是反复调参 + 串口看波形磨出来的。

学到的知识

滤波器选型的工程权衡 (FIR vs IIR、硬件 vs 软件、截止频率 vs 响应延迟); 传感器融合实战; ESP32 硬件陷阱 (strapping 引脚、ADC2/WiFi 冲突); 裸机分时调度模式; GPS NMEA/PMTK、Doppler 测速。

如果重新设计会怎么做

- 传感器融合上 1D 卡尔曼 (只融合纵向速度, 状态量少, ESP32 扛得住)。
- 校准全自动化, 删硬编码偏置, 统一静止在线校准 + 温度补偿。
- 姿态主路径加互补滤波, 解决加速时 Pitch 虚假偏移。
- 抽配置层 (所有引脚/阈值/系数集中到 `config.h`)。
- 加 host 端单元测试; 用 FreeRTOS 双核物理隔离 Web/录音与传感器融合。

下一步可以扩展什么

升级 GPS 到 NEO-M9N (GPS_UPGRADE_ANALYSIS.md 已选型, 150 元换 5-8 倍精度); 真正启用 SD 记录 + Python 分析脚本; 加磁力计 (QMC5883L) 补全 Yaw; 大倾角重力分量补偿 $a_{\text{true}} = a_{\text{meas}} - g \sin(\text{Roll})$; 网页端轨迹回放 / 成绩对比。

10 个人知识地图

由本工作区内容沉淀出的长期知识体系。带 [v] 的是本项目实际用过的。

Embedded Knowledge Map

|

- └─ MCU 基础 (ESP32)
 - └─ GPIO [v] (BOOT 按钮输入、CS 控制)
 - └─ ADC [v] (ADC1_CH6 麦克风; ADC2/WiFi 冲突坑)
 - └─ Timer + 中断 [v] (8kHz 采样定时器、IRAM_ATTR、ISR 最小化)
 - └─ Strapping 引脚 [v] (GPIO12 启动冲突坑)
 - └─ 双核 (未用, 改进方向)
- └─ 通信协议
 - └─ I²C [v] (双路总线、设备扫描、400kHz、地址 0x68/0x3C)
 - └─ UART [v] (GPS 9600、HardwareSerial2)
 - └─ SPI [v] (SD 卡 HSPI、避 VSPI 冲突)
 - └─ HTTP/WiFi [v] (SoftAP 热点、WebServer、JSON、缓存策略)
 - └─ NMEA/PMTK [v] (GPS 文本协议、Doppler 测速、配置指令)
- └─ 传感器与信号处理 [*] 本项目核心
 - └─ IMU (MPU6050) [v] (量程/DLPF 配置、6 轴)
 - └─ 滤波器 [v]
 - └─ 硬件 DLPF [v] └─ 滑动平均 (FIR) [v]
 - └─ 一阶 IIR/EMA [v] └─ 卡尔曼 (未用, 改进方向)
 - └─ 传感器融合 [v]
 - └─ 互补滤波 [v] (倾角, a=0.96) └─ 动态信心度加权 [v] └─ 阈值穿越插值 [v]
 - └─ 姿态解算 [v] (atan2 算 Roll/Pitch; Yaw 缺磁力计)
 - └─ 偏置校准 [v] (递推均值 0(1) 在线校准)
- └─ GPS / 定位
 - └─ NMEA 解析 [v] (TinyGPS++) └─ 零漂抑制 [v] (三层)
 - └─ Haversine 距离 [v] └─ HDOP/卫星质量 [v] └─ SBAS/DGPS 增强 [v]
- └─ 数据存储
 - └─ SPIFFS [v] (网页/录音/会话 JSON) └─ SD 卡 CSV [!](半成品)
 - └─ WAV 格式 [v] └─ ArduinoJson [v]
- └─ 系统架构 / 软件工程
 - └─ 裸机分时调度 [v] (millis 时间片, 无 delay) └─ 全局单例通信 [v]
 - └─ 环形缓冲区 [v] └─ 状态机 [v] (录音/测试/SD)
 - └─ 错误降级处理 [v] └─ RTOS 任务/队列 (未用, 改进方向)
- └─ 车辆动力学 (应用领域)
 - └─ G 力 (纵向/横向/垂直) [v] └─ 压弯倾角 [v] └─ 弯道半径 $r=v^2/(gG)$ [v]
 - └─ 翘头/翘尾检测 [v] └─ 加速/制动测试 [v] └─ 功率估算 (阻力模型) [v]
- └─ 机械 / 工具链
 - └─ PlatformIO [v] └─ OpenSCAD 参数化建模 [v] (cad/外壳) └─ Gerber/PCB [v]

本知识库由对工作区源码、文档、配置的递归分析生成。凡代码无法确定之处均已标注；未发现编造的功能或设计。

若代码变更，请同步更新本文档——过时的笔记比没有笔记更危险。